Foundations of Word2vec

[Word2vec]

$$J(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{j\neq 0}\log p(w_{t+j} \mid w_t)$$

1.0 Content Block

The idea of skip-gram is to represent each word with a vector, such that it is possible to predict the vectors of its neighbors using the vector of a word. The architecture is still linear-linear-softmax, the same as CBOW, but the input and the output are switched. Specifically, for each word w i {\displaystyle w_{i}} in the corpus, the one-hot encoding of the word is used as the input to the neural network. The output of the neural network is a probability distribution over the dictionary, representing a prediction of individual words in the neighborhood of w i {\displaystyle w_{i}} . The objective of training is to maximize ∑ i ∑ j ∈ N ln ■ Pr (w j + i ■ w i) {\displaystyle \sum _{i}\sum _{j\in N}\ln \Pr(w_{j+i}\mid w_{i})} . In full formula, the loss function is – ∑ i ∑ j ∈ N ln ■ e v w j + i ′ · v w i ∑ w ′ e v w ′ ′ · v w i {\displaystyle -\sum _{i}\sum _{j\in N}\ln {\frac {e^{v_{w_{j+i}}'\cdot v_{w_{i}}}}{\sum _{w'}e^{v_{w'}'\cdot v_{w_{i}}}}}} Same as CBOW, once such a system is trained, we have two trained matrices V , V ′ {\displaystyle V,V'} . Either the column vectors of V {\displaystyle V} or the row vectors of V ′ {\displaystyle V'} can serve as the dictionary. It is also possible to simply define V ′ = V ■ {\displaystyle V'=V^{\top }} , in which case there would no longer be a choice. Essentially, skip-gram and CBOW are exactly the same in architecture. They only differ in the objective function during training.

2.0 Content Block

Context window The size of the context window determines how many words before and after a given word are included as context words of the given word. According to the authors' note, the recommended value is 10 for skip-gram and 5 for CBOW.

3.0 Content Block

Assessing the quality of a model Mikolov et al. (2013) developed an approach to assessing the quality of a word2vec model which draws on the semantic and syntactic patterns discussed above. They developed a set of 8,869 semantic relations and 10,675 syntactic relations which they use as a benchmark to test the accuracy of a model. When assessing the quality of a vector model, a user may draw on this accuracy test which is implemented in word2vec, or develop their own test set which is meaningful to the corpora which make up the model. This approach offers a more challenging test than simply arguing that the words most similar to a given test word are intuitively plausible.

4.0 Content Block

top2vec Another extension of word2vec is top2vec, which leverages both document and word embeddings to estimate distributed representations of topics. top2vec takes document embeddings learned from a doc2vec model and reduces them into a lower dimension (typically using UMAP). The space of documents is then scanned using HDBSCAN, and clusters of similar documents are found. Next, the centroid of documents identified in a cluster is considered to be that cluster's topic vector. Finally, top2vec searches the semantic space for word embeddings located near to the topic vector to ascertain the 'meaning' of the topic. The word with embeddings most similar to the topic vector might be assigned as the topic's title, whereas far away word embeddings may be considered unrelated. As opposed to other topic models such as LDA, top2vec provides canonical 'distance' metrics between two topics, or between a topic and another

embeddings (word, document, or otherwise). Together with results from HDBSCAN, users can generate topic hierarchies, or groups of related topics and subtopics. Furthermore, a user can use the results of top2vec to infer the topics of out-of-sample documents. After inferring the embedding for a new document, must only search the space of topics for the closest topic vector.

5.0 Content Block

Analysis The reasons for successful word embedding learning in the word2vec framework are poorly understood. Goldberg and Levy point out that the word2vec objective function causes words that occur in similar contexts to have similar embeddings (as measured by cosine similarity) and note that this is in line with J. R. Firth's distributional hypothesis. However, they note that this explanation is "very hand-wavy" and argue that a more formal explanation would be preferable. Levy et al. (2015) show that much of the superior performance of word2vec or similar embeddings in downstream tasks is not a result of the models per se, but of the choice of specific hyperparameters. Transferring these hyperparameters to more 'traditional' approaches yields similar performances in downstream tasks. Arora et al. (2016) explain word2vec and related algorithms as performing inference for a simple generative model for text, which involves a random walk generation process based upon loglinear topic model. They use this to explain some properties of word embeddings, including their use to solve analogies.

6.0 Content Block

History During the 1980s, there were some early attempts at using neural networks to represent words and concepts as vectors. In 2010, Tomáš Mikolov (then at Brno University of Technology) with co-authors applied a simple recurrent neural network with a single hidden layer to language modelling. Word2vec was created, patented, and published in 2013 by a team of researchers led by Mikolov at Google over two papers. The original paper was rejected by reviewers for ICLR conference 2013. It also took months for the code to be approved for open-sourcing. Other researchers helped analyse and explain the algorithm. Embedding vectors created using the Word2vec algorithm have some advantages compared to earlier algorithms such as those using n-grams and latent semantic analysis. GloVe was developed by a team at Stanford specifically as a competitor, and the original paper noted multiple improvements of GloVe over word2vec. Mikolov argued that the comparison was unfair as GloVe was trained on more data, and that the fastText project showed that word2vec is superior when trained on the same data. As of 2022, the straight Word2vec approach was described as "dated". Transformer-based models, such as ELMo and BERT, which add multiple neural-network attention layers on top of a word embedding model similar to Word2vec, have come to be regarded as the state of the art in natural language processing.